

Vincenzo Rubulotta
Matricola 1000034149

Relazione Homework 2 Prova in Itinere

DSBD

1. Introduzione

Il presente elaborato illustra l'evoluzione dell'architettura a microservizi realizzata nel precedente progetto, focalizzandosi sull'incremento della robustezza, della scalabilità e del disaccoppiamento del sistema attraverso l'adozione di pattern avanzati di ingegneria del software. Pur mantenendo inalterato il nucleo funzionale originale, costituito dai servizi User Manager e Data Collector con i rispettivi database, l'infrastruttura di comunicazione è stata radicalmente ottimizzata.

In prima istanza, il precedente Gateway applicativo è stato sostituito da un server Nginx, configurato come Reverse Proxy per gestire in modo più efficiente il traffico in ingresso. L'innovazione più significativa riguarda tuttavia l'adozione di un paradigma Event-Driven supportato dal message broker Apache Kafka, che ha permesso l'introduzione di due nuovi componenti: l'Alert System e il Notifier System. Questi microservizi cooperano per gestire il monitoraggio delle metriche di volo in tempo reale: qualora i dati raccolti rilevinno valori eccedenti o inferiori alle soglie prefissate, il sistema innesca una catena di eventi asincroni che culmina con l'invio di una notifica all'utente tramite protocollo di messaggistica di posta elettronica. Infine, per garantire la continuità operativa anche in presenza di instabilità dei servizi esterni, è stato implementato il pattern Circuit Breaker all'interno del Data Collector, tutelando in tal modo le comunicazioni verso le API di OpenSky Network.

2. Schema Architetture

Nel seguente schema a blocchi è illustrata la logica del sistema preso in esame:

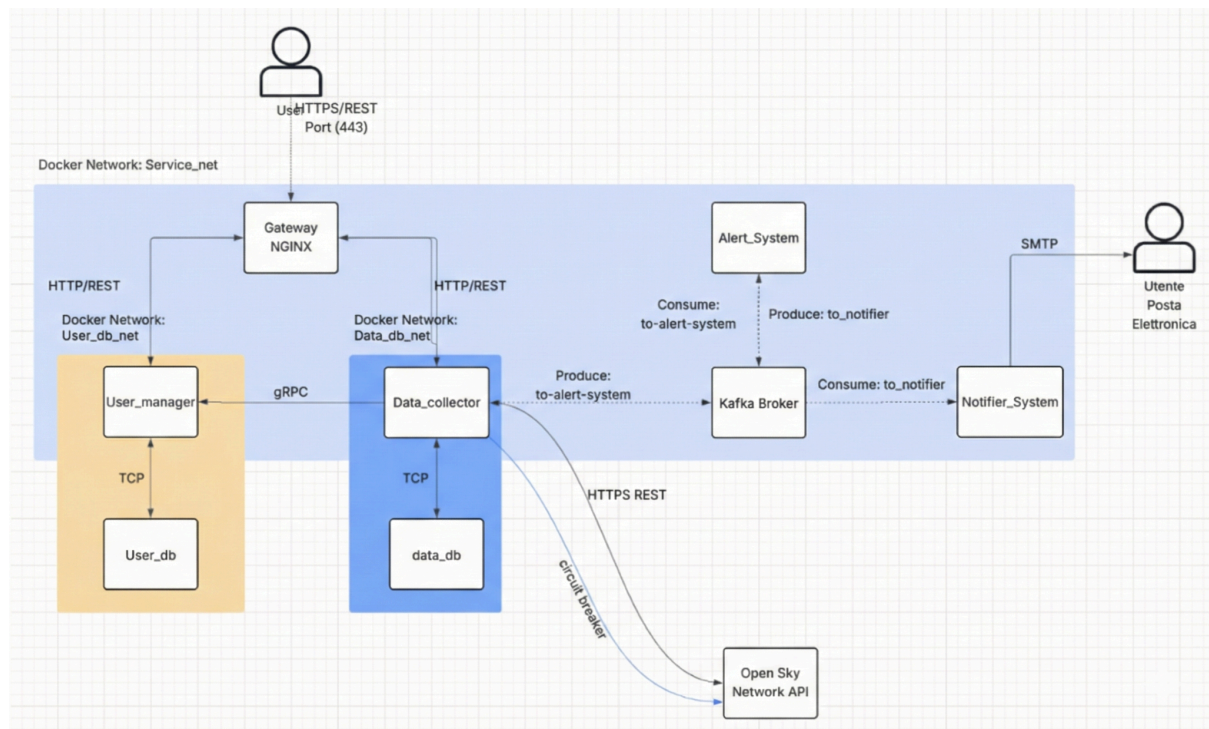


Figura 1 - Diagramma architetturale dei componenti e flussi di comunicazione

2.1 Descrizione dei componenti

Di seguito si procederà con la descrizione dei componenti principali del sistema, ognuno dei quali isolato nel proprio container per garantire modularità, con particolare attenzione alle novità introdotte rispetto alla versione precedente.

Gateway Nginx

Il Gateway Nginx sostituisce il precedente gateway applicativo e rappresenta ora l'unico punto di accesso al sistema per il client esterno sulla porta 443. Questo componente agisce come Reverse Proxy e si occupa di inoltrare le richieste HTTPS REST verso lo User Manager o il Data Collector. La sua adozione offre una gestione standardizzata del routing, disaccoppiando l'interfaccia pubblica dalla logica interna dei servizi.

User Manager

Questo microservizio mantiene il suo ruolo responsabile della gestione dei dati relativi ai vari utenti registrati al sistema, restando invariato rispetto al primo progetto. Esso continua a gestire le operazioni

di registrazione e verifica degli utenti e funge da unico punto di accesso al proprio database dedicato, garantendo l'integrità delle informazioni anagrafiche.

Data Collector

Data Collector Il secondo microservizio è responsabile della raccolta dei dati di volo, ma in questa architettura assume un ruolo decisamente più evoluto. Oltre a gestire la logica di business e le chiamate gRPC, questo componente agisce ora anche come Kafka Producer: quando vengono recuperati nuovi dati di volo, il Collector pubblica un evento sul topic to-alert-system, delegando l'analisi successiva. Inoltre, le interazioni verso le API esterne di OpenSky Network sono ora mediate da un meccanismo di Circuit Breaker, introdotto per garantire la resilienza del sistema e gestire eventuali fallimenti del servizio terzo.

Gestione Eventi e Notifiche

A supporto della nuova architettura asincrona è stato introdotto un Message Broker basato su Apache Kafka e Zookeeper, che costituisce la spina dorsale per lo scambio dei messaggi e assicura che il Data Collector non venga bloccato dalle operazioni di notifica. A valle del broker operano due nuovi microservizi: l'Alert System, che legge i dati dal topic to-alert-system per valutare il superamento delle soglie e produrre eventi di allarme, e il Notifier System. Quest'ultimo rappresenta il componente finale della catena di notifica, incaricato di consumare gli eventi dal topic to_notifier e di gestire l'invio effettivo della notifica all'utente tramite protocollo di posta elettronica SMTP.

Layer di Persistenza

Il layer di persistenza rimane strutturalmente simile alla versione precedente ed è composto dai database separati per gli utenti e per i dati di volo. Questi mantengono il loro rigoroso isolamento di rete, essendo accessibili esclusivamente tramite i rispettivi gestori, ovvero lo User Manager e il Data Collector, preservando così la sicurezza e l'indipendenza dei dati archiviati.

2.2 Topologia delle Network

Invece di utilizzare un'unica rete condivisa, il sistema è diviso in tre segmenti distinti. Queste quindi rimangono invariate rispetto alla precedente versione del progetto, solamente la rete service_net a cui sono stati aggiunti tre nuovi componenti.

La comunicazione tra i servizi principali, ovvero API Gateway, User Manager e Data Collector, Broker Kafka, Alert System, Notifier System avviene sulla service_net, una rete di tipo bridge dove lo scambio di dati utilizza il protocollo gRPC. Per la gestione dei dati, invece, sono state create due reti private e isolate: la user_db_net, che collega esclusivamente lo User Manager al suo database, e la data_db_net, che connette il Data Collector al proprio database. Questa separazione garantisce che non ci sia alcun collegamento diretto, nemmeno a basso livello, tra il Data Collector e il database degli utenti, aumentando notevolmente la sicurezza dell'intero sistema.

3. Scelte Progettuali e Motivazioni

In questa sezione vengono analizzate le decisioni architetturali critiche adottate per l'evoluzione del sistema, motivando le scelte implementative.

3.1 Adozione di Nginx

L'introduzione di Nginx in sostituzione del precedente Gateway applicativo (Python/Flask) rappresenta una delle modifiche architetturali più rilevanti, comportando un significativo refactoring nella gestione delle richieste HTTPS. Mentre nella prima versione il Gateway centralizzato fungeva da traduttore tra le chiamate JSON esterne e le procedure gRPC interne, la configurazione di Nginx come Reverse Proxy puro ha imposto la migrazione della logica di gestione degli endpoint REST direttamente all'interno dei microservizi (User Manager e Data Collector). Nonostante tale decentralizzazione, il disaccoppiamento tra il client e il sistema è stato preservato: Nginx maschera efficacemente la topologia della rete interna, esponendo un'unica interfaccia sulla porta 443 e instradando le richieste ai servizi competenti in modo trasparente.

3.2 Implementazione del Circuit Breaker

Al fine di tutelare il sistema da potenziali instabilità delle API esterne (OpenSky Network), si è optato per un'implementazione custom del pattern Circuit Breaker, evitando il ricorso a librerie di terze parti preesistenti. Tale decisione nasce dalla necessità di esercitare un controllo granulare sulla logica di transizione tra gli stati (Closed, Open, Half-Open) e sulla definizione dei parametri di timeout. Sotto il profilo implementativo, per preservare la pulizia della Business Logic del Data Collector, il Circuit Breaker è stato realizzato mediante un decoratore Python. Questa soluzione architetturale consente di incapsulare la logica di fault tolerance attorno alle funzioni che eseguono chiamate esterne in modo elegante e non intrusivo, mantenendo una netta separazione tra il codice deputato alla gestione dei dati di volo e quello responsabile della resilienza, a tutto vantaggio della leggibilità e della manutenibilità del software.

3.3 Architettura Event-Driven con Kafka

Per la gestione degli alert si è virato verso un paradigma asincrono. L'integrazione di Apache Kafka consente al Data Collector di operare come Producer puro, limitandosi alla pubblicazione dei dati di volo senza dover attendere l'esito delle operazioni successive. Questo disaccoppiamento assicura che il monitoraggio dei voli non subisca latenze dovute all'elaborazione degli allarmi o all'invio della posta elettronica, compiti ora interamente demandati ai servizi Alert System e Notifier System.

Lo scambio di informazioni si articola lungo una pipeline sequenziale gestita tramite due canali distinti. Il flusso ha origine nel Data Collector che, non appena recuperati i dati validi da OpenSky, provvede a serializzarli e pubblicarli sul topic `to-alert-system`. Su questo canale opera in ascolto l'Alert System il quale analizza i messaggi in tempo reale per verificare il rispetto delle condizioni predefinite, come ad esempio il superamento di una soglia di ritardo. Qualora venga rilevata una violazione, questo componente assume il ruolo di produttore generando un evento di allarme strutturato che viene instradato sul topic `to_notifier`. A valle della catena interviene infine il Notifier System che preleva l'evento e gestisce l'interazione con il server SMTP per il recapito della notifica finale all'utente. Questa architettura assicura una netta separazione delle responsabilità, permettendo al Data Collector di ignorare le logiche di notifica e all'Alert System di focalizzarsi esclusivamente sull'analisi dei dati, delegando interamente la gestione della comunicazione al componente finale.

3.4 Gestione delle Notifiche SMTP

Il modulo di notifica (Notifier System) implementa l'invio effettivo di messaggi di posta elettronica sfruttando le librerie standard di Python ([email.mime](#)). L'autenticazione avviene tramite protocollo SMTP sicuro, utilizzando una "App Password" dedicata per garantire che le comunicazioni provengano da un account verificato. Parallelamente, si è scelto di mantenere la strategia di testing su Postman già adottata nella precedente iterazione, inclusa la generazione automatica di indirizzi email casuali tramite Pre-request Script. Sebbene tali indirizzi non corrispondano a utenti reali, questa metodologia risulta funzionale agli obiettivi del progetto: essa permette infatti di validare l'intera pipeline di esecuzione dall'innesco dell'alert fino all'interazione con il server SMTP e di sottoporre il sistema a stress test simulando un carico variegato di utenti, indipendentemente dall'effettivo recapito del messaggio finale.

3.5 Strategia di Networking

Nonostante l'integrazione di componenti infrastrutturali complessi quali Apache Kafka, si è optato per preservare la topologia di rete originaria, articolata su tre segmenti distinti (`service_net`, `user_db_net`, `data_db_net`), evitando l'istituzione di una sottorete dedicata esclusivamente al messaging. La collocazione del Cluster Kafka all'interno della `service_net` risponde a una precisa scelta architettureale orientata alla scalabilità prospettica: considerata la natura di Event Bus del componente, è verosimile ipotizzare che, in future iterazioni del sistema, altri microservizi (come lo User Manager) possano necessitare di interagire con la coda dei messaggi. Tale configurazione predispone l'architettura a queste potenziali evoluzioni senza richiedere onerosi interventi di riconfigurazione infrastrutturale. Contestualmente, il vincolo di sicurezza prioritario l'isolamento dei Database rimane rigorosamente garantito tramite le reti dedicate, assicurando che la flessibilità introdotta a livello applicativo non comprometta in alcun modo la protezione del livello dati.

4. API e Flussi di Comunicazione

In questa sezione vengono descritti in dettaglio gli endpoint esposti dall' Gateway NGINX. Per ogni API, viene illustrato il metodo di accesso, i dati richiesti e il flusso di comunicazione interno tra i microservizi coinvolti.

4.1 Registrazione Utente

La funzionalità di registrazione è stata riprogettata per integrarsi con la nuova infrastruttura basata su Nginx. Il flusso ha inizio con l'invio di una richiesta POST contenente i dati anagrafici sulla porta 80; il gateway, operando come Reverse Proxy puro, inoltra il payload JSON direttamente allo User Manager senza effettuare alcuna conversione di protocollo. Il microservizio, che ha internalizzato la logica REST, valida la richiesta ed esegue l'inserimento dei dati nel database isolato. A seguito del salvataggio, viene restituito al client un codice HTTP 201 Created. Questa riorganizzazione sposta la responsabilità della terminazione REST sul microservizio stesso, rendendo lo User Manager un componente in grado di gestire traffico HTTP e chiamate gRPC interne.

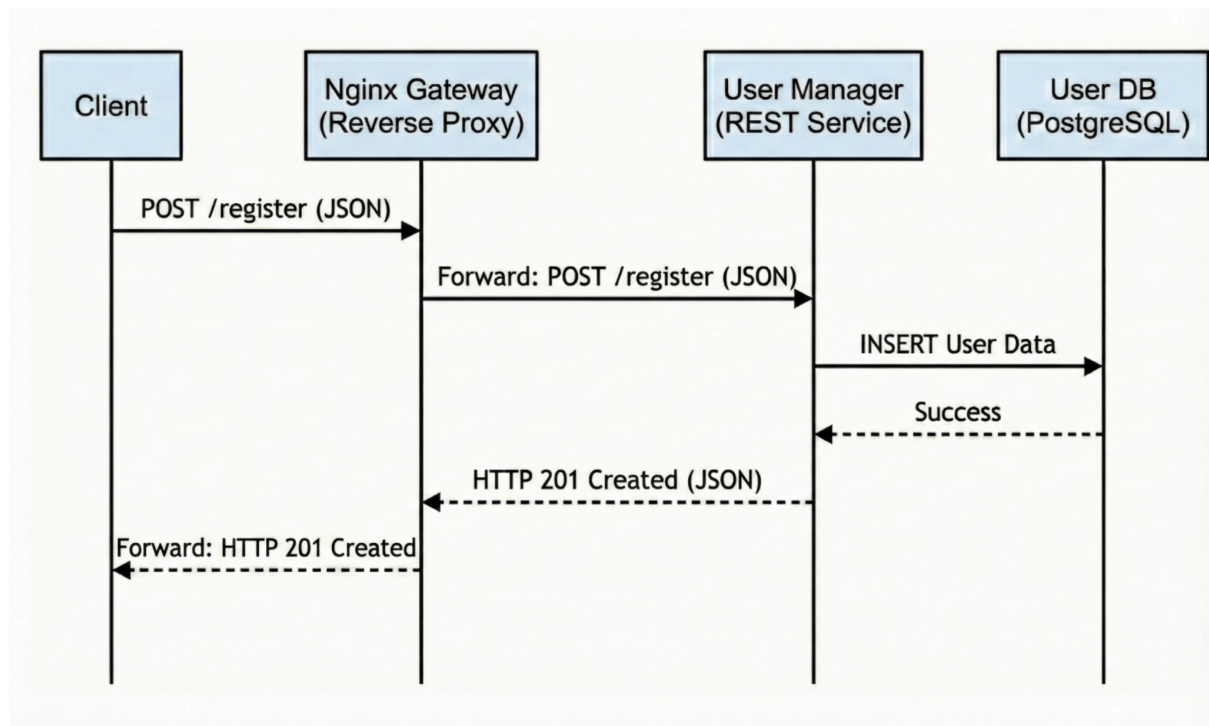


Figura 2: Diagramma di sequenza Registrazione Utente

4.2 Cancellazione Utente

La procedura di cancellazione, finalizzata alla rimozione definitiva di un profilo dal sistema, segue il medesimo paradigma architetturale introdotto con Nginx. Il flusso operativo viene innescato da una richiesta HTTPS con metodo DELETE inviata al proxy sulla porta 443, il quale provvede a instradarla trasparentemente verso il microservizio User Manager. Quest'ultimo, gestendo direttamente l'interfaccia REST, elabora l'identificativo fornito nell'URL ed esegue la query di eliminazione sul database isolato; l'esito dell'operazione viene infine comunicato al client tramite i codici di stato standard, restituendo un 200 OK in caso di successo o un 404 Not Found qualora l'utenza non sia presente, garantendo così la robustezza e la coerenza del sistema distribuito.

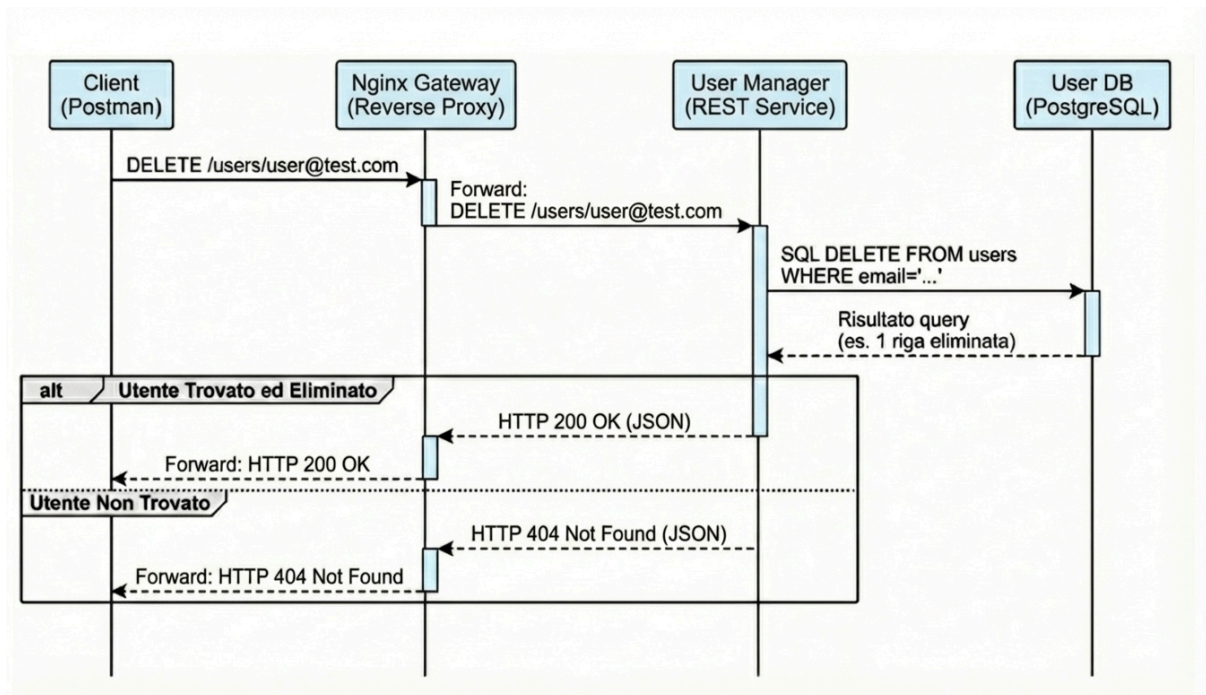


Figura 3: Diagramma di sequenza Eliminazione Utente

4.3 Verifica Esistenza Utente

Il processo si attiva con una richiesta HTTPS GET inviata al gateway Nginx, il quale agisce da pass-through inoltrando la chiamata direttamente al microservizio User Manager. Quest'ultimo, ricevuta l'interrogazione sull'endpoint REST esposto, esegue una lookup mirata sul proprio database isolato per riscontrare la presenza dell'indirizzo email specificato. L'esito della ricerca viene successivamente incapsulato in un payload JSON e restituito al client con un codice di stato 200 OK, fornendo così un meccanismo rapido per il controllo delle identità senza compromettere la sicurezza del livello dati.

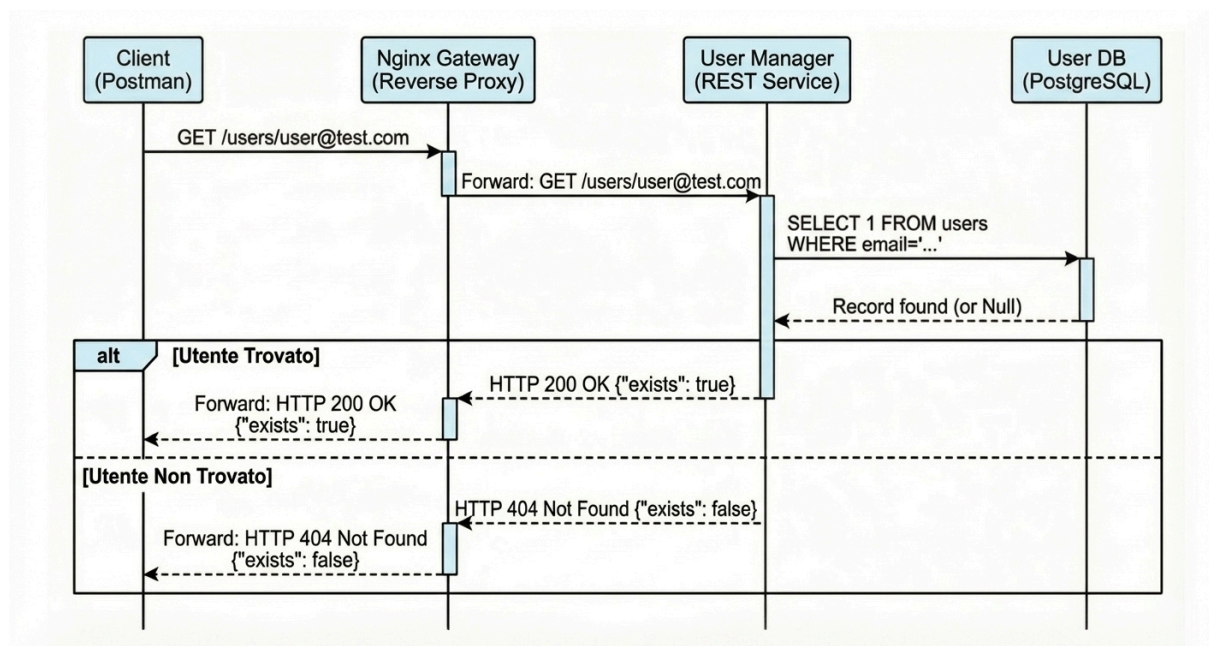


Figura 4.1: Diagramma di sequenza Verifica Esistenza Utente HTTP REST

La verifica dell'esistenza dell'utente assume un ruolo critico anche nelle logiche interne del sistema, in particolare quando il Data Collector necessita di validare una richiesta prima di procedere con l'elaborazione. In questo scenario, il componente agisce come client gRPC invocando la procedura remota CheckUserExists direttamente sullo User Manager tramite la rete di servizio interna; questa interazione, gestita tramite messaggi Protocol Buffers per garantire bassa latenza, permette di accertare la presenza dell'anagrafica nel database isolato senza violare i confini di accesso ai dati. La risposta booleana restituita consente infine al Data Collector di proseguire o interrompere il flusso operativo, mantenendo così la coerenza delle informazioni tra i due microservizi distinti.

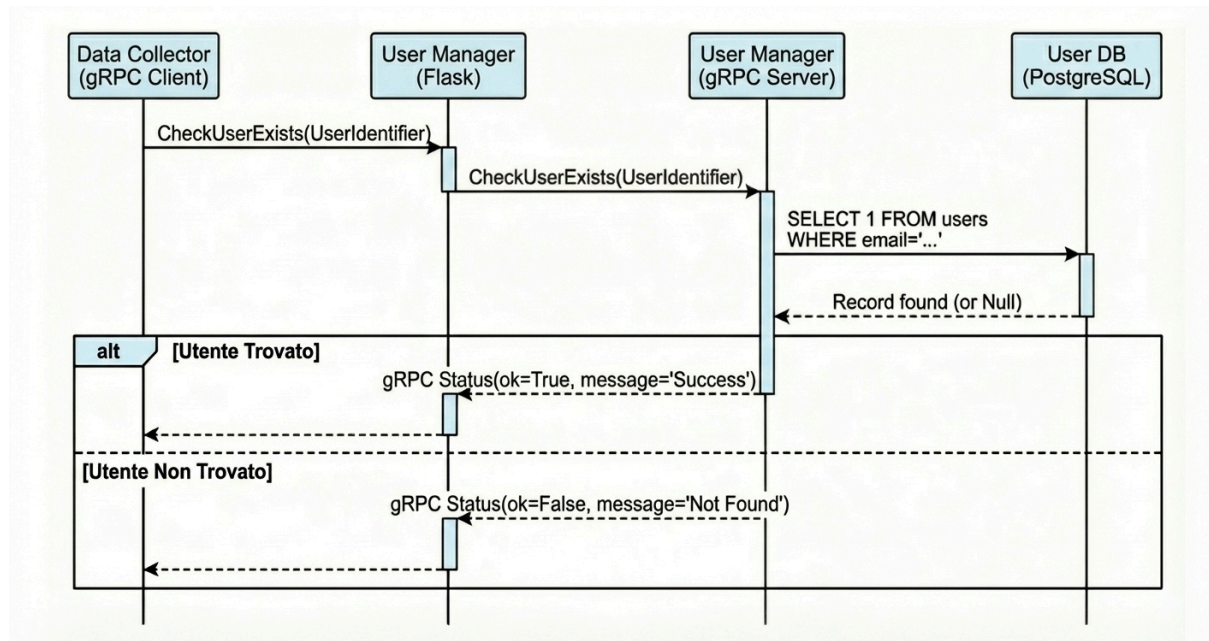


Figura 4.2: Diagramma di sequenza Verifica Esistenza Utente gRPC

4.4 Registrazione Interessi

La funzionalità di registrazione degli interessi consente all'utente di definire la lista degli aeroporti da monitorare, innescando un flusso ibrido che coinvolge entrambi i microservizi principali. Il processo ha inizio con una richiesta HTTPS POST inoltrata dal gateway Nginx direttamente al Data Collector, il quale gestisce la chiamata REST ricevendo il payload JSON contenente le preferenze. Prima di procedere alla persistenza, il componente effettua una necessaria validazione di sicurezza invocando via gRPC la procedura CheckUserExists sullo User Manager; questo passaggio assicura che la richiesta provenga da un'utenza censita senza violare l'isolamento dei database. Ottenuta la conferma, il Data Collector aggiorna le informazioni nel proprio database dedicato, sostituendo le vecchie preferenze con le nuove e confermando infine l'operazione al client con un esito positivo.

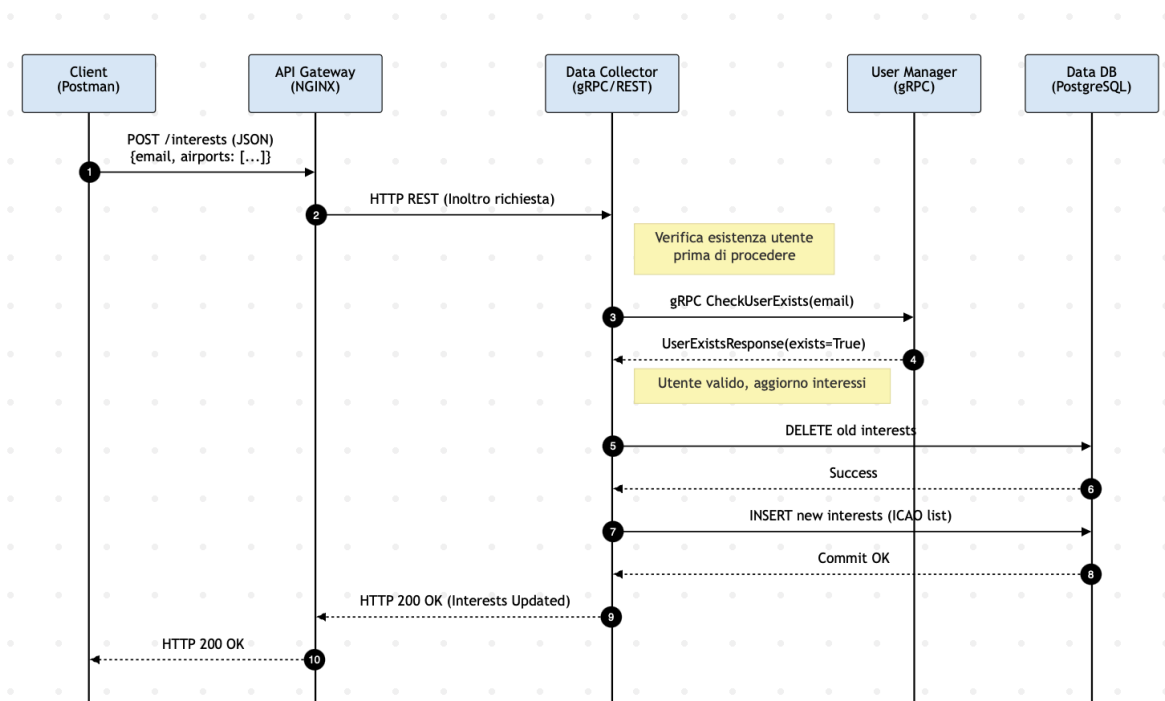


Figura 5: Diagramma di sequenza Registrazione Interessi

4.5 Recupero Dati Volo

La funzionalità dedicata al recupero puntuale dei dati storici consente all'utente di ottenere informazioni immediate su arrivi e partenze, implementando una strategia di caching volta a minimizzare le chiamate esterne. Il flusso operativo viene innescato da una richiesta HTTPS GET che il gateway Nginx inoltra trasparentemente al Data Collector; quest'ultimo provvede innanzitutto a validare l'identità del richiedente interpellando lo User Manager tramite gRPC. Successivamente viene verificata la presenza dei dati nel database locale e, solo in caso di Cache Miss, il sistema interroga l'API OpenSky Network. È importante notare che, trattandosi di una richiesta sincrona avviata dall'utente, questa operazione si limita a salvare i risultati nel database e a restituirli al client, senza attivare la pipeline di messaggistica Kafka che rimane prerogativa esclusiva del monitoraggio automatico in background. La chiamata esterna è comunque protetta dal Circuit Breaker custom per garantire la stabilità del servizio anche in caso di failure del provider dati.

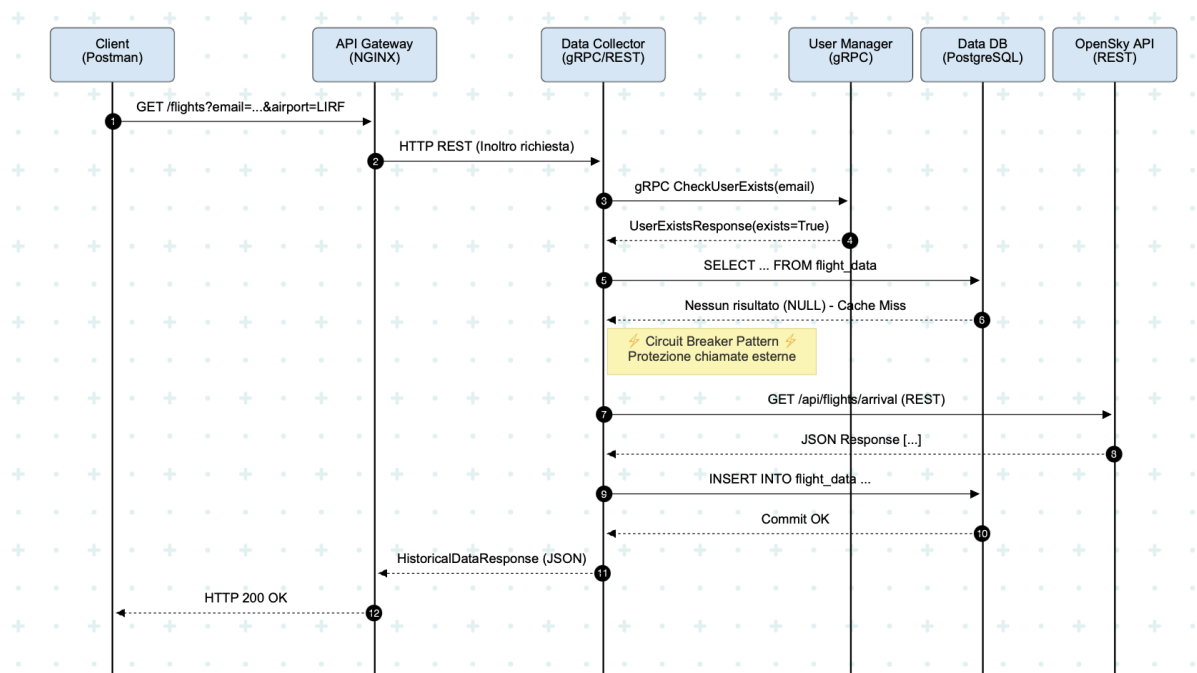


Figura 6: Diagramma di sequenza Recupero Dati Volo

4.6 Statistiche Voli

Il processo ha inizio con una richiesta HTTPS GET che il gateway Nginx, agendo come proxy puro, inoltra direttamente al microservizio Data Collector. Quest'ultimo, prima di impegnare risorse di calcolo, valida l'autorizzazione del richiedente tramite una chiamata gRPC interna verso lo User Manager; una volta confermata l'identità, il servizio esegue le query di conteggio esclusivamente sul database locale Data DB. Il sistema elabora i risultati e restituisce immediatamente la risposta JSON al client.

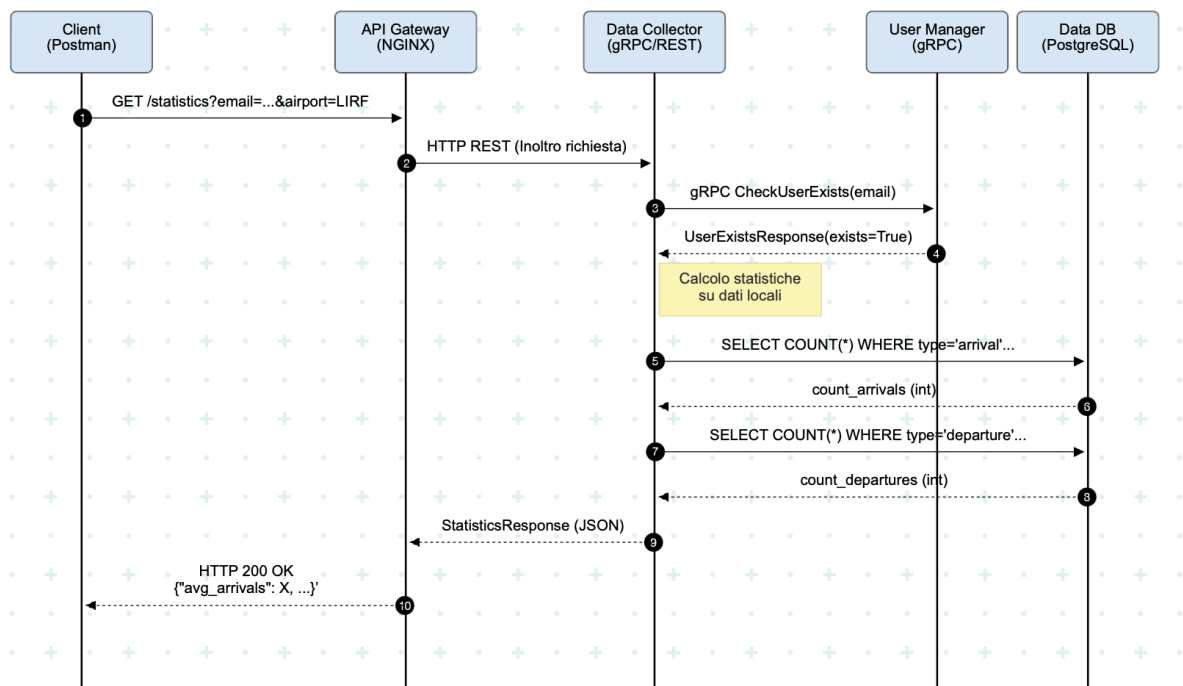


Figura 7: Diagramma di sequenza Recupero Statistiche Voli